>> Intelligent Systems – WC 2 <<

Informed and Adversarial Search

General information WC

- Post a message on Slack if you have questions: we will try to answer within 24 hours on workdays.

- In WC you will practice with exercises to understand the theory of this course
- Preparation for the exam
- !!Make sure you have paper and pen with you!!
- Prepare yourself for WC
  - Look up the slides
  - Watch videos
  - Read relevant material in the book
  - Try to make the WC exercises beforehand

Concepts to be known after WC2 (Uninformed search + Informed search)

Representation of state-space problems:

Uninformed search:

Informed search

- Hill-climbing (= local greedy search)
- Best-first search (= global greedy search)
- A
- A*
- Admissible

Adversarial search/Games

- MiniMax
- Alpha/beta pruning

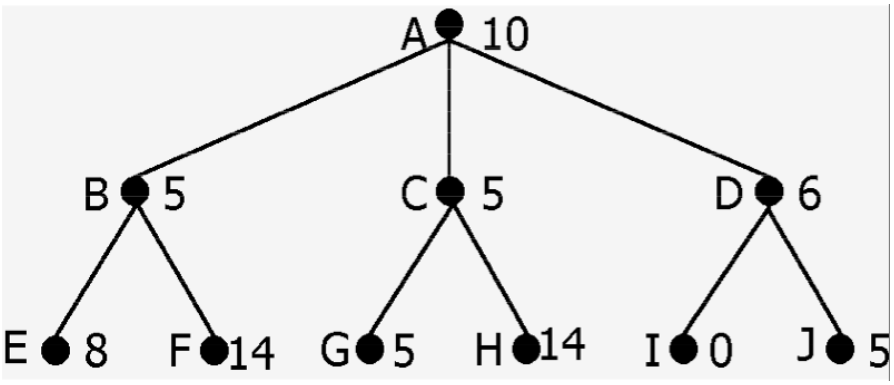
Reminder: Tree Search algorithm (pseudocode)

```
function TREE-SEARCH(problem,fringe) return a solution or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if EMPTY?(fringe) then return failure
        node ← REMOVE-FIRST(fringe)
        if GOAL-TEST[problem] applied to STATE[node] succeeds
            then return SOLUTION(node)
        fringe ← INSERT-ALL(EXPAND(node, problem), fringe)
```

Exercise 1: When do you advise to search data-driven (or 'data-directed' or 'forward searching') instead of goal-driven (or 'goal-directed' or 'backward searching')?
Data-driven when there are too many goal states, and in case goal states are not explicitly known.
You should also look at the branching factors in both directions. When this size is larger when starting from the Initial State you should, e.g., do goal directed search.

Exercises informed search

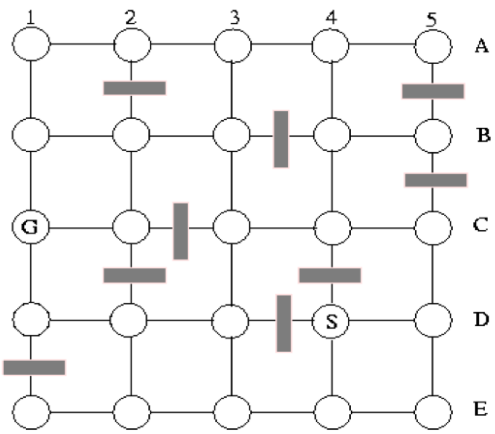
Exercise 2: Consider the following state-space, in which node I is a goal node. Furthermore, the lower the heuristic values, the better.



a) What is the order in which the nodes are searched by using **hill climbing**? Write down the letters of the nodes. Choose in alphabetical order when necessary.
AB. Once B is reached there is no node in the fringe with a higher value of h.

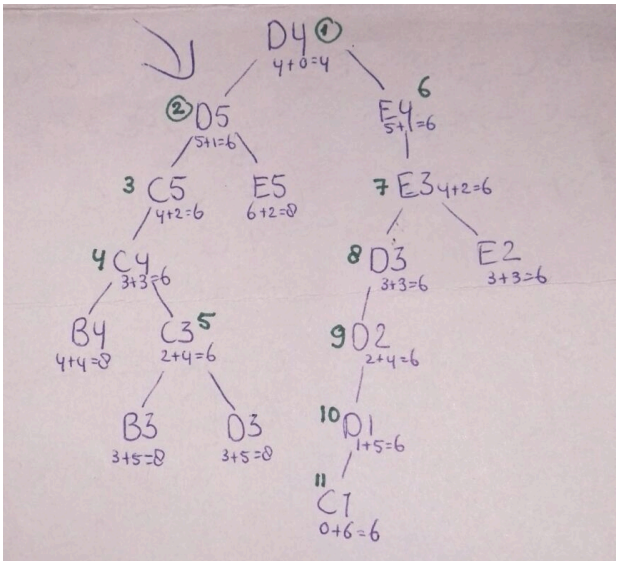
b) What is the order in which the nodes are searched by using **greedy best-first search**? Choose in alphabetical order when necessary. ABCGDI
You get the following fringes:
A -> push A
B,C,D -> push B
C,D,E,F -> push C
G,D,E,F,H -> push G
D,E,F,H -> push D
I,J,E,F,H ->push I solution found.

Exercise 3: Given the following maze:



Goal of the maze is to search a route from start position D4 (marked ‘S’) to goal position C1 (marked ‘G’).

Draw for this maze the search tree by applying a data-driven A* algorithm that uses the Manhattan distance as heuristic h(n). This means that h(n) is just an addition of the horizontal and vertical distances, without taking into account the walls. Which path will eventually be searched by A*? Number the nodes in visiting order by the algorithm. Previously searched nodes with equal or higher values can be ignored. Furthermore, write down the used heuristic value of each node. Use alphabetical order when you encounter equal values.



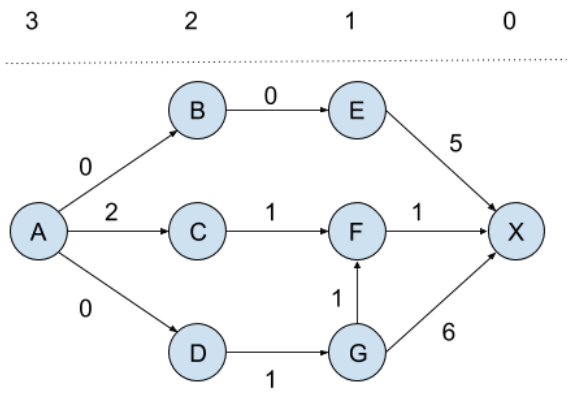
Path that will be searched by A*: D4 D5 C5 C4 C3 E4 E3 D3 D2 D1 C1
(you can emphasize the difference here again between the given answer, or the answer if the question would have been ‘What is the eventual solution path A* will find?’ D4 E4 E3 D3 D2 D1 C1).

Exercise 4: Fill in the gaps with ‘greedy, ‘admissible’, ‘underestimates’, ‘overestimates, ‘node’, ‘goal’. (not necessarily all).
A heuristic is _____ if it never _____ the costs to reach a _____.
Admissible, overestimates, goal.

Exercise 5: Which of the following search methods is/are *optimal*?

a) Breadth-first
b) Depth-first
c) Depth-first iterative deepening
d) Hill-climbing
e) Best-first
f) A
g) A*
h) Beam

Exercise 6: Consider the following search problem:

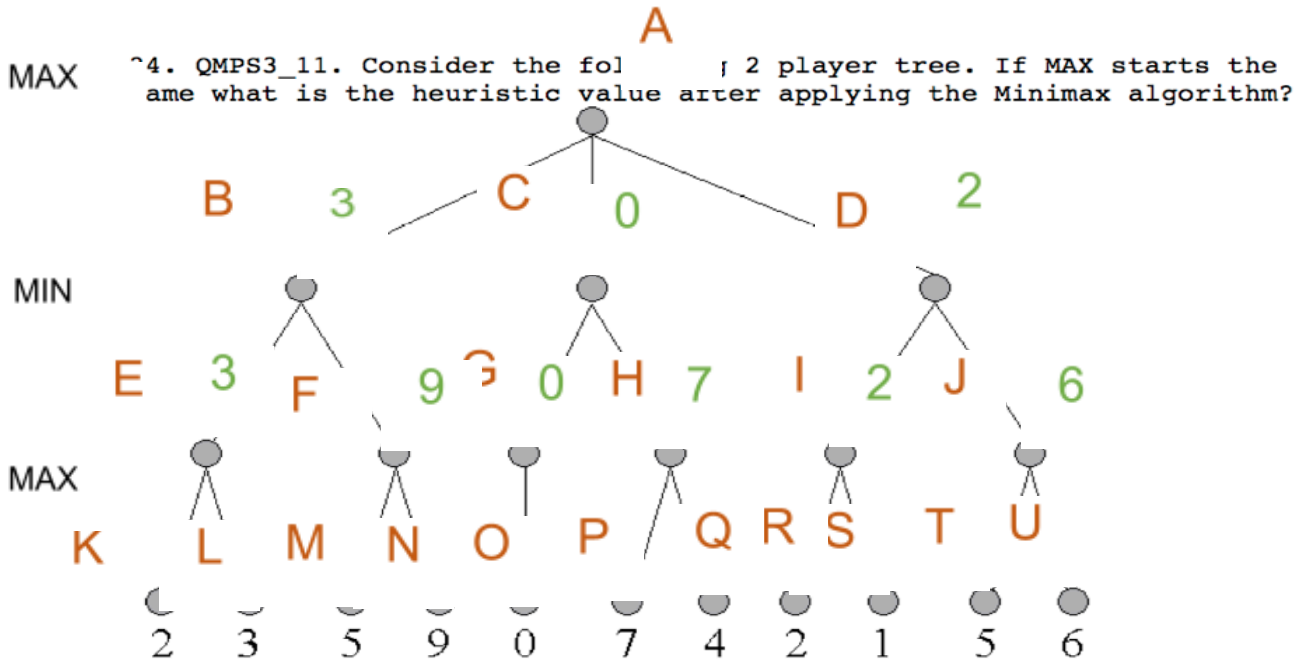


We start in A and want to reach X. The edge labels are the cost of moving from one node to another, and we want to find the path from A to X with the smallest total cost. Draw the A* search tree for this problem. The numbers above the dotted line represent the estimated cost to the goal state for all nodes below it (i.e. for nodes E, F and G, the estimated cost to the goal is 1, and the actual costs are 5, 1 and 6 respectively).
When two nodes have equal value, choose the one with the lowest letter alphabetically to expand first.
We say that a node is *added* when its evaluation function is computed. According to the algorithm from the lecture, only when a node is expanded it is checked whether it is a solution or not. A node is *expanded* when its children are added.

- Do the estimates above the dotted line lead to an admissable evaluation function (answer yes/no)? (YES, there is no way you can get from A to X than in 3 steps, and so on for all the other nodes.)
- How often is the goal state added? 3 times, when you explore E, G and F
- Which node is never expanded? C
-

Exercises adversarial search/games

Exercise 7: Consider the following 2 player game tree. If MAX starts the game, what is the heuristic value after applying the Minimax algorithm?
Intuitively, it should be easy to see that the result is 3 (nodes with values resp. 3 – 302 – 390726 from the root down)



But please try really apply the algorithm and not just follow your intuition, so that you really understand the details of how the algorithm runs. There are three functions defined in the lecture:

function MINIMAX-DECISION(*state*) returns *an action*
  inputs: *state*, current state in game
  v ← MAX-VALUE(*state*)
  return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*) returns *a utility value*
  if TERMINAL-TEST(*state*) then return EVAL (*state*)
  v ← - ∞
  for *a,s* in SUCCESSORS(*state*) do
    v ← MAX(*v*,MIN-VALUE(*s*))
  return *v*

function MIN-VALUE(*state*) returns *a utility value*
  if TERMINAL-TEST(*state*) then return EVAL (*state*)
  v ← ∞

```
for a,s in SUCCESSORS(state) do
  v ← MIN(v,MAX-VALUE(s))
return v
```

Start with MINIMAX-Decision(A)

```
MAX-VALUE(A)
        TERMINAL-TEST(A) is false, so
        v <- - ∞      (note, this is a new v not yet defined)
        for a,s in SUCCESSORS(A) do <- the successors are B,C,D
            v <- MAX(v,MIN-VALUE(s))
        return v
```

we need to calculate MIN-VALUE(B), MIN-VALUE(C) and MIN-VALUE(D) and then take the Max to calculate v. Draw a new box

```
MIN-VALUE(B)
        TERMINAL-TEST(B) is false, so
        v <- ∞          (note, this is a new v, so not the one before)
        for a,s in SUCCESSORS(B) do <- the successors are E,F
            v <- MIN(v,MAX-VALUE(s))
        return v
```

we need to calculate MAX-VALUE(E) and MAX-VALUE(F). Draw a new box

```
MAX-VALUE(E)
        TERMINAL-TEST(A) is false, so
        v <- -∞          (note, this is a new v, so not the one from before)
        for a,s in SUCCESSORS(E) do <- the successors are K,L
            v <- MAX(v,MIN-VALUE(s))
        return v
```

we need to calculate MIN-VALUE(K) and MIN-VALUE(L). Draw a new box

```
MIN-VALUE(K)
        TERMINAL-TEST(K) is true, so
        return EVAL(K) = 2
```

```
MIN-VALUE(L)
        TERMINAL-TEST(L) is true, so
        return EVAL(K) = 3
```

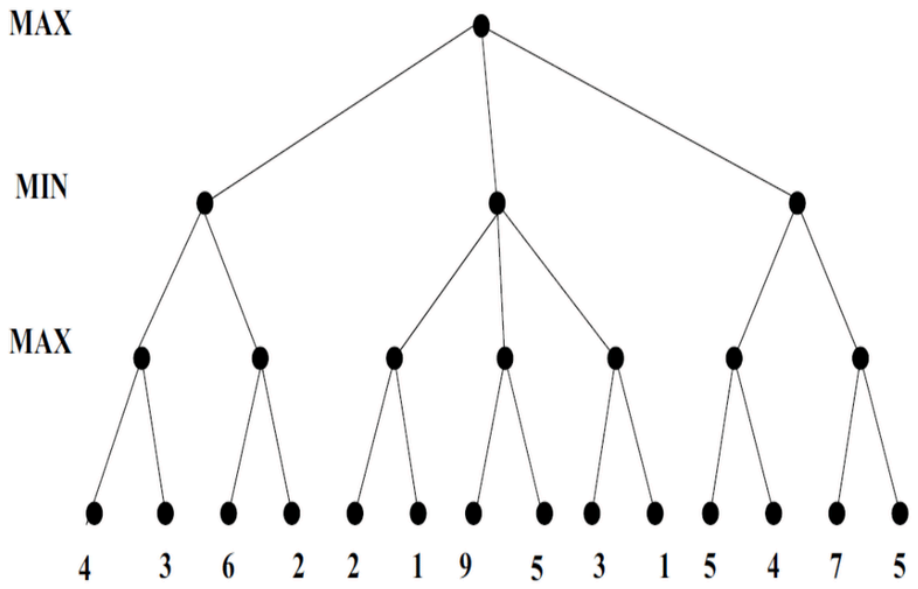
```
This means we can calculate MAX-VALUE(E)
            v <- MAX(-∞,2,3)
        return 3
```

In order to calculate MIN-VALUE(B), we also have to calculate MAX-VALUE(F), which we do exactly do like MAX-VALUE(E), with recursive calls to MIN-VALUE(M) and MIN-VALUE(N). MAX-VALUE(F) is returns v = MAX(-∞,5,9) =9. Now we need to calculate MIN-VALUE(C) (check the green box).

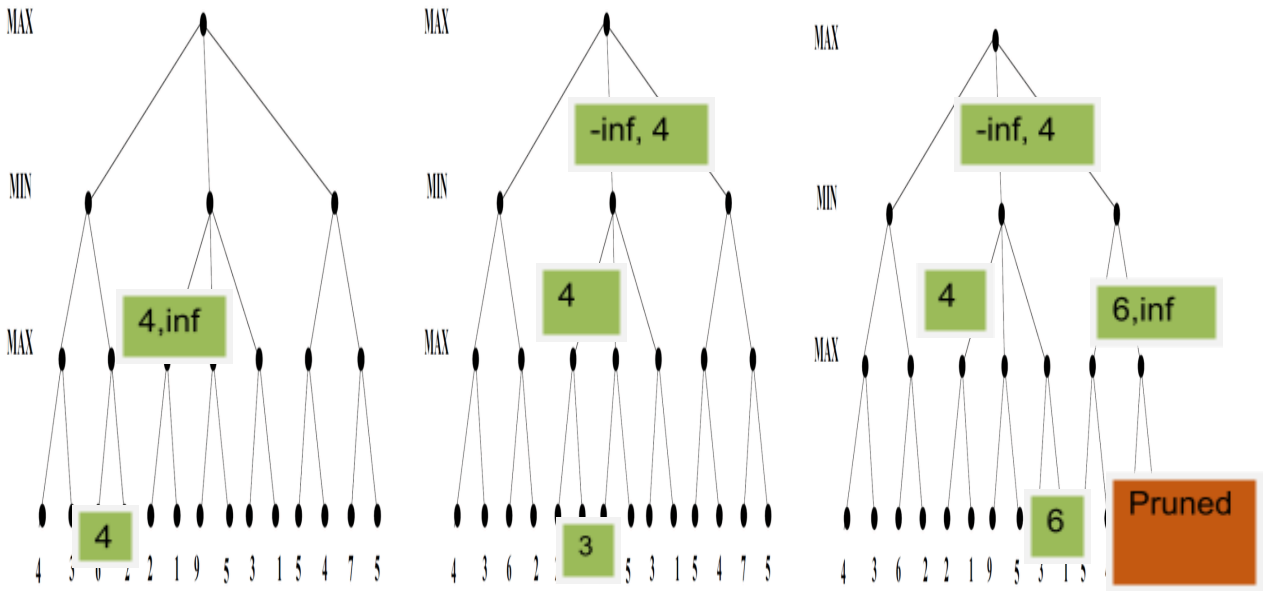
```
MIN-VALUE(C)
        TERMINAL-TEST(C) is false, so
        v <- ∞          (note, this is a new v, so not the one before)
       for a,s in SUCCESSORS(C) do <- the successors are G, H
            v <- MIN(v,MAX-VALUE(s))
       return v
```

we need to calculate MAX-VALUE(G) and MAX-VALUE(H), and so on….

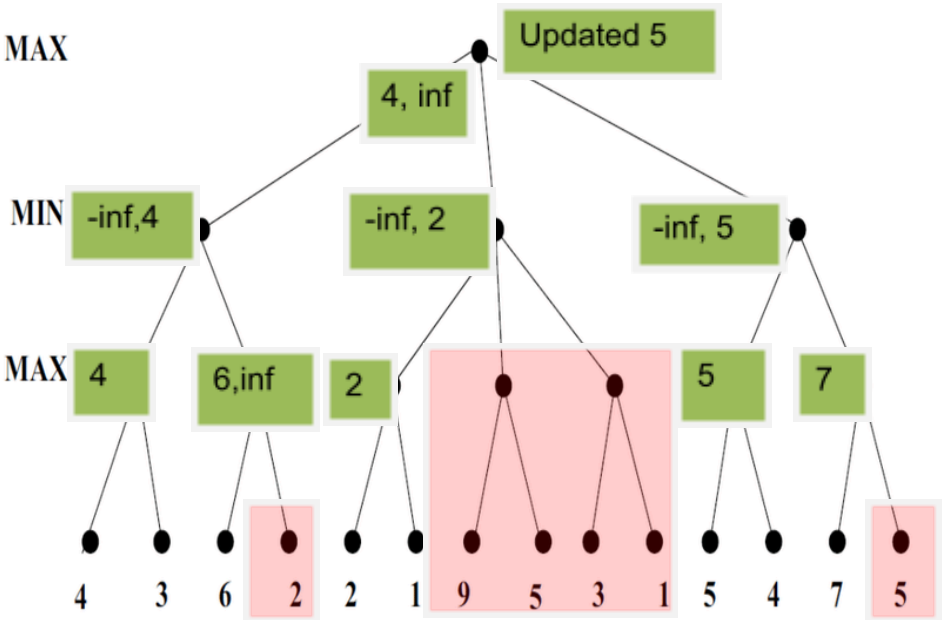
Exercise 8: Consider the following search tree that contains heuristic values at the leaves. The player on the move is depicted at each level. Which nodes can be ignored if we apply alpha-bèta pruning if the goal for Min is to have the smallest possible value, and for Max to have the largest possible value.



Let us see what happens step by step:



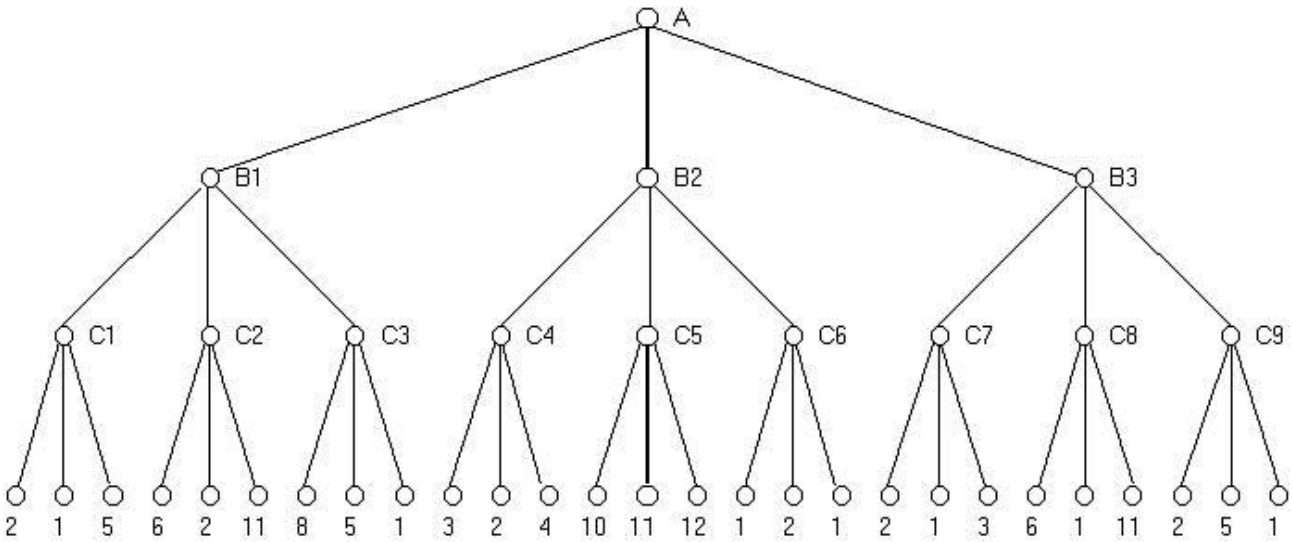
This leads to the following full tree with the pruned parts in red:

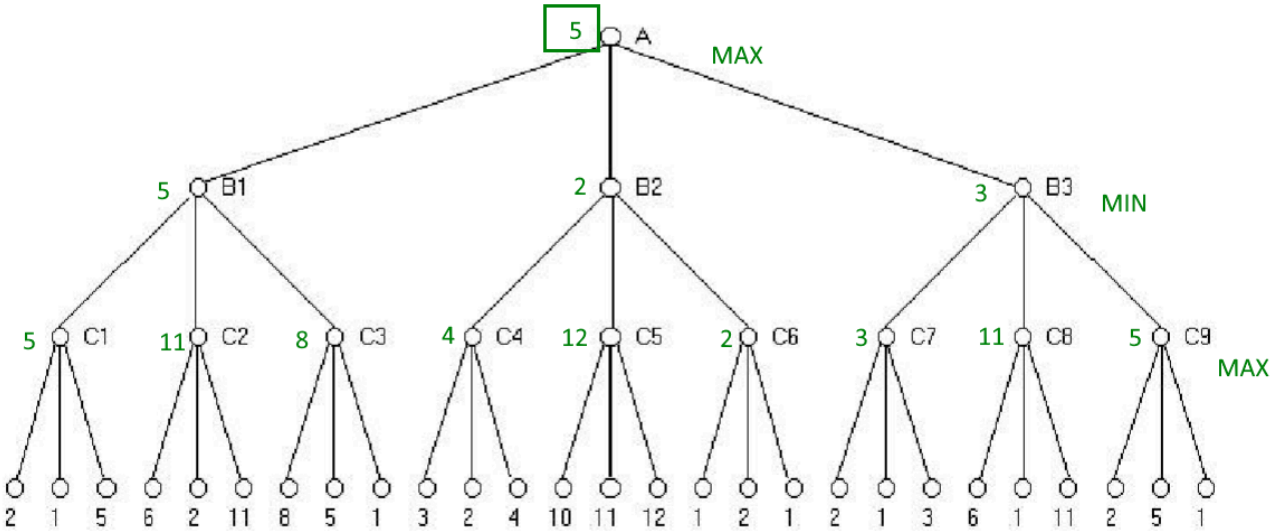


Exercise 9: True or false? If MAX plays optimally using the MinMax algorithm, and is allowed to start, it will always find a winning strategy.

Give them time to think about, but the answer is FALSE. Even though MinMax is optimal , which means that it will find a solution if there is one, it cannot always win (there might be games in Schnapsen where the cards you get are so bad that you cannot win. So optimality does not imply to win all the time.

Exercise 10: Consider the following MinMax tree. Use the MinMax algorithm to compute what the heuristic value is at position A for MAX when both players play optimally. Assume that MAX makes the first move.




5
A
MAX
5
B1
2
B2
3
B3
MIN
5
C1
11
C2
8
C3
4
C4
12
C5
2
C6
3
C7
11
C8
5
C9
MAX
2 1 5 6 2 11 8 5 1 3 2 4 10 11 12 1 2 1 2 1 3 6 1 11 2 5 1